2007-07-24

# Running own code on the Infrant ReadyNAS

Filed under Projects ReadyNAS

The Infrant ReadyNAS NV is a Sparc (Leon) powered NAS. It's not really cheap, but, well, I've got it somehow cheap. It featueres 4 SATA channels and Gigabit ethernet, unfortunately it runs a heavily modified Linux Version. Now, Infrant (of course!) complies to the GPL by releasing a 2.4.20-based kernel every now and then, of course every time with an undocumented set of changes under the same filename. Did I forget to tell you that some kernel releases are missing important files (lp_code, lp_data for example was missing in the "2.0" release), usually compile only with an ancient gcc (2.95.3, you name it!) and still contain a lot of binary-only modules?

However, you're not supposed to compile the kernel, it's really just for GPL compliance. I, however, always wanted to write some SPARC assembler code, so I've analyzed the system a bit. It boots from an unencrypted NAND flash, then reads the kernel+initrd, decrypt them, and boot them. The system then looks for HDDs, and, if it has found an uninitialized one, it will install the "core" system (means: rootfs) on it. After that, it will boot from that rootfs.

There are some hacks to gain root access (I also developed another method, but it doesn't give you any advantages over the documented one, so I'll spare the details), or you could manually calculate the root password using a hash of the MAC address, the version number and a shared secret (...no comment on that one.).

But that wasn't enough, I wanted to run my own code directly. So I've checked the ports on the backside of the device, and voila, the middle port (this is for the ReadyNAS NV, other devices might differ) is a 9600 TTL serial port. The pinout is GND,RX,TX,VCC (from reset switch to the other 6-pinport). If you keep pressing 'i' on bootup, you get into a (small) console.

Now, from this console you can tftp-boot (tftp 00:09:34:11:22:33 10.0.0.203 10.0.0.1 /tftpboot/hello 0x80040000, where 00:09:34:11:22:33 is the mac address which will be used, 10.0.0.203 the client IP address, 10.0.0.1 the server IP address, /tftpboot/

hello the binary, and 0x80004000 the loading address). With “go 0x80040000” you can then jump to memory. To make it short: it doesn’t work that way. Because the tftp’ed binary must be encrypted using the hardware encryption of the device. Now, there is a way to encrypt binaries with a tool running on the device, but I didn’t wanted that. So I developed a short way to disable the encryption:

tftp 00:09:34:11:22:33 10.0.0.203 10.0.0.1 /tftpboot/rn_nops 0x87832660

(note that this only works for iboot 1.00a030.)

After that, the decryption will be disabled, and you can just upload any binary, for example my hello-world.tgz. Of course you need a proper toolchain.

A full boot looks like the following:

```
Loading ...
Welcome to iboot 1.00a030
built 04:29 2006-10-26
ECC ON
OC[7f]
ECC ON
Reason:
Help:
tftp MAC IP IPsvr file mem
nand
usb
ics v12
ICS reg val
pci v17
reason x
go mem
auto
iboot> iii
?
iboot>
iboot> tftp 00:09:34:11:22:33 10.0.0.203 10.0.0.1 /tftpboot/rn_nops 0x87832660

OK
iboot> tftp 00:09:34:11:22:33 10.0.0.203 10.0.0.1 /tftpboot/hello 0x80040000
```

```
OK
iboot> go 0x80040000
Hello 31337 world!
30000000:  10 80 00 62 01 00 00 00 01 00 00 00 01 00 00 00
30000020:  01 00 00 00 01 00 00 00 01 00 00 00 01 00 00 00
30000040:  01 00 00 00 01 00 00 00 01 00 00 00 01 00 00 00
30000060:  01 00 00 00 01 00 00 00 01 00 00 00 01 00 00 00
30000080:  01 00 00 00 01 00 00 00 01 00 00 00 01 00 00 00
300000a0:  10 80 01 07 a1 50 00 00 01 00 00 00 01 00 00 00
300000c0:  10 80 01 15 a1 50 00 00 01 00 00 00 01 00 00 00
300000e0:  40 00 00 39 ac 10 20 0a 40 00 00 37 ac 10 20 0d
30000100:  40 00 00 35 ac 10 20 6c 40 00 00 33 ac 10 20 6f
30000120:  40 00 00 31 ac 10 20 6f 40 00 00 2f ac 10 20 70
30000140:  05 0c 00 00 c0 20 80 00 c2 00 80 00 80 80 40 00
30000160:  02 bf ff fe 01 00 00 00 40 00 00 27 ac 10 20 2a
30000180:  c4 00 80 00 01 00 00 00 ad 30 a0 1c ac 0d a0 0f
300001a0:  40 00 00 21 ac 05 a0 30 ad 30 a0 18 ac 0d a0 0f
300001c0:  40 00 00 1d ac 05 a0 30 ad 30 a0 14 ac 0d a0 0f
300001e0:  40 00 00 19 ac 05 a0 30 ad 30 a0 10 ac 0d a0 0f
```

I'm sorry by the way to not provide any photos. My digital camera is broken (no joke!). And I'm too lazy to use any other camera lying around, like from my N800. (which is, based on the image quality, good for you, believe me). Just imagine a ReadyNAS NV board lying on my desktop, with an external ATX power supply (the original one exploded(!) on the first powerup due to a screw(!) inside the PSU, as my post-mortem diagnosis showed. Though I was promised a replacement from the distributor, I've never got it, and I got sick of asking again and again. If you want to use an ATX supply for your ReadyNAS, you *need* to remove R204 and R205 on the backplane/power board. They shortcut -5V to GND or something like this), with some wires sticking into the named serial port, going to the Hoodie Hacker AdapterDLP-TXRX-G USB-to-TTL-serial adapter.

---

© debugmo.de

# Recent Posts

Blogs

FJITA: The Project that Just Wasn't Meant To Be

Custom Firmware for DJI Mini 2

Autoliv NV2 Nightvision Camera teardown

My Scanning Electron Microscope

A Hacker's Look at Dieselgate

OpenVizsla OV3 - Hello, World!

OpenVizsla OV3 - FPGA design

OpenVizsla OV3 - Hardware

What's Inside: Tektronix DPO5034

Real Life Statistics

xvcd - The Xilinx Virtual Cable Daemon

What's Inside: Hilti PD-30

Almost Secure

What's Inside: Metz 50 AF-1 N

"if you call that hacking, then we embrace that.",...

The Last Piece

Encrypted Links

Retro Hacking

politics

Scope pr0n

${stupid-syscall-02(*)-joke-here}

And the wimp of the year award goes to...

Probe Hacking, Part 2

Probe Hacking, Part 1